

ISMP 3D Anatomy — Documentacion de API

Backend NestJS + Prisma + PostgreSQL

Base URL: **http://localhost:3001**

AUTH /auth

POST /auth/login

Acceso	Publico — sin token
Recibe	{ "email": "admin@ismp.edu.ar", "password": "admin123" }
Devuelve	{ token, user: { id, name, email, role, carreraId } } — devuelve 401 si la cuenta esta PENDIENTE

Busca el usuario por email, compara el password con el hash almacenado y verifica que estado = ACTIVO. Genera un JWT firmado.

GET /auth/me

Acceso	Cualquier rol autenticado — Bearer token
Recibe	Header: Authorization: Bearer <token>
Devuelve	{ id, name, email, role, carreraId, createdAt }

Devuelve los datos del usuario dueño del token.

USERS /users

POST /users/register

Acceso	Publico — sin token
Recibe	{ "name", "email", "password" (min 6 chars), "carreraId"? }
Devuelve	{ message, activationCode } — codigo de 8 caracteres en mayusculas

Crea un estudiante con estado PENDIENTE. El alumno debe pasar el activationCode a un admin para activar su cuenta. No puede loguearse hasta ser activado.

POST /users/activate

Acceso	ADMIN — Bearer token
Recibe	{ "code": "A3F9BC2D" }
Devuelve	{ message: "Usuario Juan activado correctamente" }

Busca el usuario por activationCode, lo pasa a estado ACTIVO y elimina el codigo para que no pueda reutilizarse.

GET /users/search?search=xxx

Acceso	DOCENTE, ADMIN — Bearer token
Recibe	Query param: search (nombre o email parcial)
Devuelve	Array de { id, name, email, carreraId } — busqueda insensible a acentos y mayusculas

Busca estudiantes en la tabla User usando unaccent() de PostgreSQL. Util para encontrar un alumno antes de consultar sus intentos.

GET	/users
Acceso	ADMIN — Bearer token
Recibe	Query params opcionales: role (ADMIN DOCENTE ESTUDIANTE), carreraId
Devuelve	Array de usuarios sin passwordHash
GET	/users/:id
Acceso	ADMIN — Bearer token
Recibe	Path param: id del usuario
Devuelve	{ id, name, email, role, carreraId, createdAt } — 404 si no existe
POST	/users
Acceso	ADMIN — Bearer token
Recibe	{ "name", "email", "password", "role" (ADMIN DOCENTE ESTUDIANTE), "carreraId"? }
Devuelve	Usuario creado — a diferencia de /register, no genera codigo ni pasa por aprobacion
PATCH	/users/:id
Acceso	ADMIN — Bearer token
Recibe	Cualquier campo del usuario (todos opcionales). Si viene password la hashea.
Devuelve	Usuario actualizado
DELETE	/users/:id
Acceso	ADMIN — Bearer token
Recibe	Path param: id del usuario
Devuelve	{ message: "Usuario eliminado correctamente" } — 404 si no existe

QUESTIONS /questions

GET	/questions
Acceso	Cualquier rol autenticado — Bearer token
Recibe	Query params opcionales: materiaId, unidad (int), formato (MULTIPLE TRUEFALSE IDENTIFICATION LABELING)
Devuelve	Array de preguntas con opciones, respuesta correcta, materias y autor

Lista el banco de preguntas. Sin filtros devuelve todas. Consulta las tablas Question y QuestionMateria.

GET	/questions/:id
Acceso	Cualquier rol autenticado — Bearer token
Recibe	Path param: id de la pregunta

Devuelve	Pregunta completa con materias y autor — 404 si no existe
POST	/questions
Acceso	DOCENTE, ADMIN — Bearer token
Recibe	{ "question", "options": [...4 opciones], "correct": 0-3, "explanation"?, "formato", "materias": [{ materialId, unidad }] }

Devuelve Pregunta creada con sus relaciones a materias — el autorId se toma del token
Crea una pregunta en el banco. Inserta en Question y crea los registros pivot en QuestionMateria.

PATCH	/questions/:id
Acceso	DOCENTE (solo sus preguntas), ADMIN — Bearer token
Recibe	Mismos campos que POST pero todos opcionales. Si viene materias reemplaza todas las relaciones.
Devuelve	Pregunta actualizada

DELETE	/questions/:id
Acceso	DOCENTE (solo sus preguntas), ADMIN — Bearer token
Recibe	Path param: id de la pregunta
Devuelve	{ message: "Pregunta eliminada correctamente" }

ATTEMPTS /attempts

POST	/attempts/start
Acceso	Cualquier rol autenticado — Bearer token (típicamente ESTUDIANTE)
Recibe	{ "materialId", "unidad": 0 (toda la materia) o 1-N, "cantidad": N }
Devuelve	{ attemptId, total, numero: 1, pregunta: { id, texto, opciones, formato } } — sin la respuesta correcta

Selecciona N preguntas aleatorias del banco para la materia/unidad indicada. Crea un registro en Attempt y los registros en AttemptQuestion (snapshot de preguntas asignadas). Devuelve la primera pregunta.

POST	/attempts/:id/answer
Acceso	Cualquier rol autenticado — Bearer token (debe ser el dueño del intento)
Recibe	{ "questionId", "selected": 0-3 } — Path param: attemptId
Devuelve	Si hay mas preguntas: { tipo:"continua", correcta, correcta_era, explicacion, numero, total, pregunta }. Si es la ultima: { tipo:"finalizado", correcta, correcta_era, explicacion, resultado: { nota, correctas, total } }

Valida que la pregunta pertenezca al intento y no haya sido respondida. Guarda el AnswerLog con snapshot de la pregunta. Si es la ultima, cierra el intento calculando la nota (0-100).

GET	/attempts/me
Acceso	Cualquier rol autenticado — Bearer token

Recibe	Query param opcional: materialId
Devuelve	Array de intentos completados del usuario con datos de la materia — 404 si no tiene ninguno

GET /attempts/student

Acceso	DOCENTE, ADMIN — Bearer token
Recibe	Query params: studentId (obligatorio), materialId y carreraId (obligatorios para DOCENTE, opcionales para ADMIN)
Devuelve	{ estudiante: { id, name, email }, attempts: [...] } — 404 si el alumno no tiene intentos

Para DOCENTE verifica que la materia pertenezca a su carrera antes de devolver los datos.

GET /attempts

Acceso	DOCENTE, ADMIN — Bearer token
Recibe	Query param opcional: materialId (obligatorio para DOCENTE, opcional para ADMIN)
Devuelve	Array de intentos completados con usuario y materia — ordenados por fecha desc

DOCENTE sin materialId recibe 403. DOCENTE con materialId verifica que la materia pertenezca a su carrera.

GET /attempts/:id

Acceso	DOCENTE, ADMIN — Bearer token
Recibe	Path param: id del intento
Devuelve	Intento completo con usuario, materia y todos los answerLogs (respuestas del alumno con snapshots)

DOCENTE solo puede ver intentos de materias de su carrera. Devuelve 403 si intenta acceder a otra materia.